



Traveloop

Personalized Travel Planning Made Easy

Hackathon Documentation Roadmap — Updated v2

A complete, structured guide walking the team from problem statement to delivery: vision, scope, architecture, database design, feature breakdown, API contracts, UI/UX flow, milestones, testing, and submission checklist.

Project	Traveloop
	Multi-city travel itinerary planner
Stack	React + TanStack Start, Tailwind, Lovable Cloud (Postgres, Auth, Storage)
	3 – 5 members
Duration	Hackathon (24 – 48 hrs)
	https://link.excalidraw.com//65VNwvy7c4X/22o30WE3bE4

■ What changed in v2

- Section 5 – Feature Catalog: added F15 Saved Destinations (missing from v1)
- Section 7 – Database: added saved_destinations table; aligned with Profile screen requirements
- Section 8 – API Contract: added updateProfile() and manageSavedDestinations() endpoints
- Section 9 – UI/UX: added i18n / locale note for language preference (Profile screen requirement)
- Section 10 – Roadmap: split P5 to flag checklist/notes complexity; added buffer note
- Section 13 – Risks: added Auth/Session failure risk row
- Section 14 – Submission checklist: added missing Pitch Deck checkbox

1. Vision & Mission

Vision

Become a personalized, intelligent and collaborative platform that transforms how people plan and experience travel — empowering users to dream, design, and organize multi-city trips with ease.

Mission (Hackathon)

Build a user-centric, responsive web app that simplifies multi-city travel planning by letting users add stops, explore activities, estimate budgets, visualize timelines, and share plans.

2. Problem Statement

Design and develop a complete travel planning application where users can:

- Create customized multi-city itineraries
- Assign travel dates, activities and budgets
- Discover activities and destinations through search
- Receive cost breakdowns and visual calendars
- Share their plans publicly or with friends

The application must demonstrate proper use of a relational database to store and retrieve trips, stops, activities, and expenses, with a dynamic UI that adapts to each user's flow.

3. Scope & Deliverables

3.1 In-Scope (MVP)

- Authentication (email/password)
- Trip CRUD with cover photo
- Itinerary builder: stops, dates, activities, reorder
- City & activity search with filters
- Auto budget breakdown with charts
- Packing checklist and trip notes
- Public shareable itinerary URL
- Saved destinations per user profile

3.2 Stretch Goals

- Admin analytics dashboard
- Collaborative trip editing
- Map view of itinerary
- Export itinerary as PDF
- Multi-language / locale support

3.3 Final Deliverables

- Working hosted web app (preview + published URL)
- GitHub repo with README and setup steps
- ER diagram + schema migrations
- Demo video (2 – 3 min) and pitch deck

- This documentation roadmap

4. User Personas & Core Journeys

4.1 Personas

- The Explorer – plans long multi-city trips, needs structure and budget control.
- The Weekender – short focused trips, needs speed and inspiration.
- The Sharer – publishes itineraries for friends and followers.

4.2 Primary Journeys

- Sign up → Dashboard → Create Trip → Add stops/activities → View budget → Share.
- Browse public itinerary → Copy Trip → Customize → Save.
- Open trip → Manage packing checklist → Add journal notes during trip.
- Profile → Manage saved destinations → Start new trip from saved city.

5. Feature Catalog ■ UPDATED

F15 (Saved Destinations) is newly added to align with the Profile/Settings screen requirement.

ID	Feature	Priority	Summary
F01	Login / Signup	Must	Email + password, validation, forgot password.
F02	Dashboard / Home	Must	Recent trips, plan-new-trip CTA, recommended cities, budget highlight.
F03	Create Trip	Must	Name, dates, description, optional cover photo.
F04	My Trips	Must	List view of trips with edit/view/delete.
F05	Itinerary Builder	Must	Add stops, assign dates & activities, reorder cities.
F06	Itinerary View	Must	Day-wise / city-grouped timeline with toggle.
F07	City Search	Must	Search cities with country, cost index, popularity.
F08	Activity Search	Must	Filter by type/cost/duration, add to stop.
F09	Budget & Cost Breakdown	Must	Charts by category, daily averages, over-budget alerts.
F10	Packing Checklist	Should	Per-trip categorized checklist with reset.
F11	Public Itinerary	Should	Read-only public URL + Copy Trip + share buttons.
F12	Profile / Settings	Should	Edit profile, language preference, delete account.
F13	Trip Notes / Journal	Should	Per-trip / per-stop notes with timestamps.
F14	Admin / Analytics	Could	Trips, top cities, engagement, user mgmt.

F15  NEW	Saved Destinations	Should	Save cities to profile for quick reuse; view/remove saved destinations.
--	--------------------	--------	---

6. System Architecture

6.1 High-Level Diagram

```
[ Browser (React + TanStack Start) ]  
  |  
  v  
[ Server Functions / API Routes ]  
  |  
  v  
[ Lovable Cloud – Postgres + Auth + Storage ]
```

6.2 Layers

- Presentation: React, Tailwind, shadcn/ui, TanStack Router.
- State / Data: TanStack Query for server state, Zod for validation.
- Backend: Server functions for typed RPC, REST endpoints for public/share routes.
- Database: PostgreSQL via Lovable Cloud, with Row Level Security per user.
- Storage: Lovable Cloud Storage for trip cover photos.

7. Database Design ■ UPDATED

7.1 Core Entities

- users (id, email, name, avatar_url, language_preference, created_at)
- trips (id, user_id, name, description, start_date, end_date, cover_url, is_public, share_slug)
- cities (id, name, country, cost_index, popularity, lat, lng)
- stops (id, trip_id, city_id, arrival_date, departure_date, order_index)
- activities (id, name, category, cost_estimate, duration_hours, city_id, image_url)
- stop_activities (id, stop_id, activity_id, scheduled_at, notes, custom_cost)
- expenses (id, trip_id, category, amount, label, day)
- checklist_items (id, trip_id, category, label, is_packed)
- trip_notes (id, trip_id, stop_id NULL, body, created_at)
- **saved_destinations (id, user_id, city_id, saved_at) ♦ NEW – supports F15 & Profile screen**

7.2 Relationships

- User 1 — N Trips
- Trip 1 — N Stops → each Stop links to one City
- Stop N — M Activities (via stop_activities)
- Trip 1 — N Expenses / Checklist items / Notes
- User N — M Cities (via saved_destinations) ♦ NEW

7.3 Suggested Indexes

- trips(user_id), stops(trip_id, order_index), stop_activities(stop_id)
- cities(name) GIN trigram for search; activities(city_id, category)
- saved_destinations(user_id) ♦ NEW

8. API / Server Function Contract ■ UPDATED

Two endpoints added in v2 (marked ♦).

Method	Path / Function	Purpose
POST	/auth/signup	Create account
POST	/auth/login	Authenticate user
GET	getTrips()	List my trips
POST	createTrip()	Create trip
GET	getTrip(id)	Trip with stops + activities
PATCH	updateTrip(id)	Edit trip
DELETE	deleteTrip(id)	Remove trip
POST	addStop(tripId)	Add stop with city + dates
PATCH	reorderStops(tripId)	Change stop order
POST	attachActivity(stopId)	Attach activity to stop
GET	searchCities(q)	Filtered city search
GET	searchActivities(q)	Filtered activity search
GET	getBudget(tripId)	Aggregated cost breakdown
GET	/api/public/trip/:slug	Public read-only itinerary
POST	copyTrip(slug)	Clone public trip
PATCH ⚡	updateProfile()	Edit name, avatar, language preference, delete account
GET/POST/DELETE ⚡	manageSavedDestinations()	List, save, or remove cities from user's saved destinations

9. UI / UX Guidelines ■ UPDATED

- Mobile-first responsive layout; primary breakpoints 360 / 768 / 1024 / 1280.
- Design tokens in `src/styles.css` (oklch); semantic colors only.
- Typography: distinctive display font for hero, refined sans for body.
- Motion: framer-motion for hero and stop reorder; keep interactions snappy.
- Empty states for every list (no trips, no stops, no notes, no saved destinations).
- Loading skeletons + optimistic updates on builder actions.
- i18n / locale: store `language_preference` on user record; wrap UI strings in a translation helper even if only one language ships for MVP — this future-proofs the stretch goal. ♦ NEW

10. Hackathon Roadmap (Timeline) ■ UPDATED

P5 is restructured: checklist and notes each have their own DB tables and non-trivial UI — budget an extra 2 hrs buffer if the team is small.

Phase	Hours	Goals	Owner
P0 – Kickoff	0 – 2	Finalize scope, repo setup, design system, Lovable Cloud enabled	All
P1 – Foundations	2 – 8	Auth, DB schema + seed cities/activities, Dashboard shell	Backend + 1 FE
P2 – Trip Core	8 – 18	Create Trip, My Trips, Itinerary Builder (stops + activities)	Frontend
P3 – Discovery	18 – 26	City & Activity search, filters, attach to stop	Full team
P4 – Insights	26 – 34	Budget breakdown + charts, itinerary view (timeline/calendar)	FE + Data
P5 – Extras ♦	34 – 42	Packing checklist, notes, public share + copy, saved destinations — allow 2 hr buffer	Frontend
P6 – Polish	42 – 46	Empty states, animations, responsive QA, SEO, error pages	All
P7 – Demo	46 – 48	Seed demo data, record video, finalize deck, publish	All

11. Team Roles & Responsibilities

- Tech Lead / Architect – schema, server functions, code reviews.
- Frontend Lead – design system, routing, builder & itinerary view.
- Frontend Engineer – dashboards, search, settings, polish.
- Data / Backend – seed datasets (cities, activities), budget aggregation.
- Designer / PM – mockups, copy, demo video, pitch deck.

12. Testing & QA

- Unit tests on budget aggregator and date utilities.
- Integration test on createTrip → addStop → attachActivity flow.
- Manual QA matrix per feature on Chrome / Safari / mobile widths.
- Lighthouse pass for Performance, Accessibility, SEO > 90.
- Seed demo account with one rich trip for judges.

13. Risks & Mitigations ■ UPDATED

Risk	Impact	Mitigation
Scope creep	High	Lock MVP after P1; stretch only after P5
Schema rework mid-hack	High	Freeze ER diagram before P2
Sparse city/activity data	Med	Pre-seed JSON dataset (50 cities, 200 activities)
Budget calc edge-cases	Med	Single source-of-truth aggregator + tests
Demo fails live	High	Pre-record 2-min walkthrough video as backup
Auth / session failure ♦ NEW	High	Test login/signup end-to-end immediately after P1; keep a seeded demo account logged-in as fallback

14. Submission Checklist ■ UPDATED

- ☐ Hosted preview URL live
- ☐ Public GitHub repo with README, setup, env example
- ☐ ER diagram in /docs
- ☐ Demo account credentials in submission notes
- ☐ 2 – 3 min demo video uploaded
- ☐ **Pitch deck (problem, solution, demo, tech, team) ♦ was missing in v1**
- ☐ This documentation included in /docs

End of roadmap — happy hacking! Build for clarity first, polish second, and tell a clear story in the demo.